

SIREN: Implicit Neural Representations with Periodic Activation Functions

A Complete Walkthrough: Original Paper · Reproduction · Improvements

Course	Deep Learning — Assignment 2 & 3
Institution	FAST-NUCES, School of Computing
Instructor	Dr. Zohair Ahmed
Group	Group ID09
Members	Valeena Afzal (25I-8023) · Maryam Zafar (25I-8033) · Laiba Noor (25I-8035)
Original Paper	Sitzmann et al., NeurIPS 2020 arXiv:2006.09661
Code Repository	https://github.com/vsitzmann/siren

1. Original Paper Walkthrough

1.1 What is the Paper About?

Implicit Neural Representations (INRs) are neural networks that represent signals as continuous functions of spatial or temporal coordinates. Instead of storing a discrete grid of pixels or samples, an INR learns a function that maps coordinates to signal values — for example, mapping (x, y) pixel coordinates to RGB colors for an image.

The paper proposes SIREN (Sinusoidal Representation Networks) — the key insight being to replace the standard ReLU activation function with a periodic sine function. This simple change has profound consequences for the network's ability to represent signals and their derivatives.

1.2 The Core Problem with ReLU Networks

Most INRs at the time used ReLU-based MLPs. ReLU networks have a critical limitation:

- ReLU is piecewise linear — the network is literally a collection of linear pieces
- The second derivative of ReLU is ZERO everywhere — it cannot model curvature
- Cannot represent fine high-frequency details in natural signals
- Cannot be supervised on derivatives (gradients, Laplacians) of the signal
- Tanh and softplus are smooth but their derivatives are poorly behaved

1.3 The SIREN Solution

SIREN proposes using the sine function as the activation for every layer:

$$\phi_i(x) = \sin(\omega_0 * W_i * x + b_i)$$

where $\omega_0 = 30$ is a frequency scaling factor applied to all layers. The key mathematical property that makes SIREN special is:

Highlight

Any derivative of a SIREN is itself a SIREN. The derivative of sin is cos, which is a phase-shifted sine — $\sin(x + \pi/2)$. This means SIREN can accurately represent not just the signal but also its gradients, Laplacians, and higher-order derivatives — something fundamentally impossible with ReLU networks.

1.4 The Critical Initialization Scheme

The paper derives a principled initialization scheme that is ESSENTIAL for SIREN to work. Without it the network fails completely. The initialization preserves the distribution of activations through all layers:

Layer	Weight Distribution	Purpose
First layer	$U(-1/n, 1/n)$	Input spans multiple sine periods over $[-1,1]$
Hidden layers	$U(-\sqrt{6/n}/\omega_0, \sqrt{6/n}/\omega_0)$	Preserve arcsine activation distribution
Output layer	Linear (no activation)	Allow arbitrary output range

omega_0	30 for all layers	Scale frequencies to match signal content
---------	-------------------	---

The theory behind this: pushing uniform input through a sine gives an arcsine distribution. With the proposed initialization, each layer's input is Gaussian distributed and output is arcsine distributed — this pattern is preserved through ALL layers regardless of depth, preventing vanishing/exploding gradients.

1.5 Experiments in the Paper

The paper demonstrates SIREN across 6 major experiments, each showcasing a different capability that ReLU networks cannot match:

Experiment	What it does	Key metric	SIREN vs ReLU
Image Fitting	Represent cameraman image as continuous function	PSNR	SIREN ~50 dB vs ReLU ~25 dB
Audio Fitting	Represent raw audio waveform	MSE	SIREN 1.1e-5 vs ReLU 2.5e-2
Video Fitting	Represent video as 3D spatiotemporal function	PSNR	SIREN 29.90 dB vs ReLU 25.12 dB
Poisson Equation	Reconstruct image from ONLY its gradients	PSNR	SIREN 32.91 dB, ReLU fails
SDF Learning	Learn 3D signed distance function from point clouds	Visual quality	SIREN captures fine detail
Helmholtz/Wave SDE	Solve complex PDEs with boundary conditions	MSE vs grid solver	SIREN 7.9e-6 vs others fail

1.6 Why Each Experiment Matters

Image Fitting (Figure 1 in paper):

This is the flagship experiment. Five architectures fit the cameraman image: ReLU, Tanh, ReLU+Positional Encoding, RBF-ReLU, and SIREN. The paper shows not just the reconstructed image (row 1) but also the gradients (row 2) and Laplacians (row 3) of the learned function. ReLU's Laplacian is completely zero — it cannot represent any curvature. SIREN is the ONLY architecture that accurately represents all three rows. Hardware: NVIDIA Quadro RTX 6000 (24GB). Steps: 15,000.

Audio Fitting (Supplementary Table 3):

Audio waveforms are highly periodic — ideal for SIREN. Two clips tested: Bach Cello Suite (7 seconds, 44,100 Hz) and a Counting clip (12 seconds). ReLU completely fails to represent the waveform structure. SIREN achieves MSE of 1.1e-5 on Bach — orders of magnitude better. The input domain is scaled to [-100, 100] instead of [-1,1] to account for the high sampling rate.

Video Fitting (Figure 2 in paper):

A video is treated as a 3D function mapping (x, y, t) to RGB. The paper uses a cat video (300 frames, 512x512) and bikes sequence (250 frames, 272x640). SIREN achieves 29.90 dB mean PSNR, outperforming ReLU by ~5 dB. Architecture uses 1024 hidden units (larger than image/audio) to handle the 3D input space. Training takes ~15 hours on Titan X GPU.

Poisson Equation (Figure 3 in paper):

The most striking experiment — the network is NEVER shown the actual image values, only its gradients or Laplacians. SIREN successfully reconstructs the image from derivatives alone (PSNR 32.91 dB from

gradients). ReLU cannot do this at all because its second derivative is zero everywhere. This demonstrates that SIREN can solve inverse problems defined by differential equations.

2. Assignment 2: Reproduction of Results

2.1 What We Reproduced

We reproduced three experiments from the paper: image fitting, audio fitting, and video fitting using the official GitHub repository at <https://github.com/vsitzmann/siren>

2.2 Hardware Setup

Component	Paper	Our Setup
GPU	NVIDIA Quadro RTX 6000	NVIDIA RTX 3050 Laptop
VRAM	24 GB	4 GB
CUDA	11.x	12.4
OS	Linux	Windows 11
Python	3.7-3.8	3.10
PyTorch	Older	2.x (cu124)

2.3 Experimental Setup

Parameter	Image	Audio	Video
Architecture	5-layer MLP, 256 units	5-layer MLP, 256 units	5-layer MLP, 1024 units
Activation	Sine (omega=30)	Sine (omega=30)	Sine (omega=30)
Optimizer	Adam	Adam	Adam
Learning Rate	1e-4	5e-5	1e-4
Training Steps	10,000 (paper: 15,000)	9,000	5,000 (paper: 100,000)
Input Dimension	2 (x, y)	1 (time)	3 (x, y, t)
Output Dimension	1 (grayscale)	1 (amplitude)	3 (RGB)
Dataset	Camera 512x512	Bach 7s + Counting 12s	Bikes 272x640 250fr
Input Domain	[-1, 1]	[-100, 100]	[-1, 1]
Metric	PSNR (dB)	MSE	PSNR (dB)
Training Time	~7 hours	~5 hours	~9.5 hours

2.4 Compatibility Fixes Required

The repository was written for older library versions. Several fixes were required:

- `skimage.measure.compare_ssim` removed — replaced with `skimage.metrics.structural_similarity`

- skimage.measure.compare_psnr removed — replaced with skimage.metrics.peak_signal_noise_ratio
- Laplacian computation in utils.py caused CUDA OOM on 4GB GPU — disabled with torch.cuda.empty_cache()
- write_image_summary argument mismatch for RBF/NeRF models — fixed partial() call in train_img.py
- ModuleNotFoundError for skvideo — installed scikit-video via pip
- FFmpeg not found for video loading — manually installed and added to PATH
- numpy.float deprecated in skvideo — downgraded numpy to 1.23.5
- Conda environment corruption from ffmpeg DLL conflicts — rebuilt environment from scratch

2.5 Reproduced Results

Image Fitting Results:

Model	Paper PSNR	Our PSNR	Difference
SIREN	~50 dB	40.37 dB	-9.63 dB
ReLU P.E. (NeRF)	~38 dB	31.11 dB	-6.89 dB
RBF-ReLU	~29 dB	23.50 dB	-5.50 dB
ReLU	~25 dB	23.99 dB	-1.01 dB
Tanh	~26 dB	21.91 dB	-4.09 dB

Model ranking matches paper exactly: SIREN > ReLU P.E. > RBF ≈ ReLU > Tanh

Audio Fitting Results:

Architecture	Bach MSE (Paper)	Bach MSE (Ours)	Counting MSE (Paper)	Counting MSE (Ours)
SIREN	1.101e-5	~0 (converged)	3.816e-4	5.0e-4
ReLU	2.504e-2	2.48e-2	7.466e-3	7.5e-3

Audio results reproduced with exceptional fidelity — ReLU within 1% of paper values

Video Fitting Results:

Model	Paper PSNR	Our PSNR	Paper Steps	Our Steps	Status
SIREN	29.90 dB	30.70 dB	100,000	1,809	EXCEEDED paper!
ReLU	25.12 dB	19.81 dB	100,000	1,958	Below (fewer steps)

SIREN exceeded paper PSNR by 0.80 dB despite only 1.8% of the training steps — demonstrating remarkable fast convergence

2.6 Reproducibility Analysis

Aspect	Paper	Ours	Reason
Training Steps (Image)	15,000	10,000	Time constraints
Training Steps (Audio)	9,000	9,000 / 1,676*	*Counting SIREN stopped early

Training Steps (Video)	100,000	5,000	GPU memory + time
Image PSNR (SIREN)	~50 dB	40.37 dB	Fewer steps + 4GB VRAM
Laplacian Computation	Fully computed	Disabled	CUDA OOM on 4GB GPU
Hardware	RTX 6000 24GB	RTX 3050 4GB	No high-end hardware access
Video SIREN PSNR	29.90 dB	30.70 dB	Fast convergence of SIREN

2.7 Why Results Differ from Paper

- Training steps: Paper used 15,000 for images; we used 10,000. SIREN still improving at 10,000 steps — explains the PSNR gap.
- Hardware: 6x less VRAM (4GB vs 24GB) forced disabling Laplacian computation which may affect gradient flow.
- RBF model: 1024-dimensional input needs much more VRAM — only completed 1,000 steps explaining lower PSNR.
- Software versions: Newer PyTorch/scikit-image have minor numerical differences.
- Audio: Excellent match — ReLU within 1%, SIREN converges to near-zero MSE confirming the paper's claims.
- Video: SIREN actually exceeded paper due to efficient convergence even at low step count.

3. Core Implementation Code

3.1 SIREN Architecture (modules.py)

The core of SIREN is the Sine activation class with dynamic omega support and the SingleBVPNet which wraps the FCBlock:

```
class Sine(nn.Module): def __init__(self, omega_0=30): super().__init__() self.omega_0 = omega_0 def forward(self, input): return torch.sin(self.omega_0 * input) def sine_init(m): # Hidden layer initialization with torch.no_grad(): if hasattr(m, 'weight'): num_input = m.weight.size(-1) m.weight.uniform_(-np.sqrt(6/num_input)/30, np.sqrt(6/num_input)/30) def first_layer_sine_init(m): # First layer initialization with torch.no_grad(): if hasattr(m, 'weight'): num_input = m.weight.size(-1) m.weight.uniform_(-1/num_input, 1/num_input) class SingleBVPNet(MetaModule): def __init__(self, out_features=1, type='sine', in_features=2, mode='mlp', hidden_features=256, num_hidden_layers=3, **kwargs): super().__init__() self.mode = mode self.net = FCBlock(in_features=in_features, out_features=out_features, num_hidden_layers=num_hidden_layers, hidden_features=hidden_features, outermost_linear=True, nonlinearity=type) def update_omega(self, new_omega): for module in self.modules(): if isinstance(module, Sine): module.omega_0 = new_omega def forward(self, model_input, params=None): coords = model_input['coords'].clone().detach().requires_grad_(True) output = self.net(coords) return {'model_in': coords, 'model_out': output}
```

3.2 Image Training Script (train_img.py)

```
# Key parts of the image training pipeline img_dataset = dataio.Camera() coord_dataset = dataio.Implicit2DWrapper( img_dataset, sidelength=512, compute_diff='all') dataloader = DataLoader(coord_dataset, shuffle=True, batch_size=1, pin_memory=True) model = modules.SingleBVPNet(type='sine', mode='mlp', sidelength=(512,512)) model.cuda() # Loss: simple MSE on pixel values loss_fn = partial(loss_functions.image_mse, None) optimizer = torch.optim.Adam(model.parameters(), lr=1e-4) for step in range(10000): model_input, gt = next(iter(dataloader)) model_input = {k: v.cuda() for k, v in model_input.items()} gt = {k: v.cuda() for k, v in gt.items()} model_output = model(model_input) losses = loss_fn(model_output, gt) train_loss = sum(losses.values()) optimizer.zero_grad() train_loss.backward() optimizer.step()
```

3.3 Fixes Applied to utils.py

```
# Fix 1: Deprecated scikit-image API # OLD (broken): ssim = skimage.measure.compare_ssim(p, trgt, multichannel=True) psnr = skimage.measure.compare_psnr(p, trgt, data_range=1) # NEW (fixed): ssim = skimage.metrics.structural_similarity( p, trgt, channel_axis=-1, data_range=1) psnr = skimage.metrics.peak_signal_noise_ratio( p, trgt, data_range=1) # Fix 2: Laplacian OOM - in write_image_summary(): # OLD (causes OOM on 4GB GPU): img_laplace = diff_operators.laplace( model_output['model_out'], model_output['model_in']) # NEW (disabled to save VRAM): torch.cuda.empty_cache() # img_laplace computation skipped # Fix 3: Missing ground_truth_img argument for RBF/NeRF: # OLD: summary_fn = partial(utils.write_image_summary, image_resolution) # NEW: img = img_dataset[0] summary_fn = partial(utils.write_image_summary, image_resolution, img)
```


4. Assignment 3: Proposed Improvements

4.1 Overview

For Assignment 3 we proposed and implemented two improvements to SIREN for image fitting: (I) Cosine Annealed Omega Scheduling (CAOS-SIREN) and (II) a modified loss function. Both experiments were run on the same cameraman image as A2 for direct comparison.

4.2 Improvement I — CAOS-SIREN: Adaptive Omega Scheduling

Motivation:

Standard SIREN uses fixed $\omega_0=30$ throughout training. We hypothesized that starting with lower frequencies to capture global structure and gradually annealing to higher frequencies for fine detail would improve representation quality — inspired by curriculum learning principles and the FINER paper (CVPR 2024) which showed frequency support at initialization is a key bottleneck.

Method:

We implemented Cosine Annealed Omega Scheduling (CAOS) where ω_0 follows:

$$\omega(t) = \omega_{\min} + (\omega_{\max} - \omega_{\min}) * (1 - \cos(\pi * t / T)) / 2$$

Implementation code:

```
# In modules.py – make Sine accept dynamic omega class Sine(nn.Module): def __init__(self, omega_0=30): super().__init__() self.omega_0 = omega_0 # mutable def forward(self, input): return torch.sin(self.omega_0 * input) def update_omega(self, new_omega): for module in self.modules(): if isinstance(module, Sine): module.omega_0 = new_omega # In training loop – update every 500 steps def caos_omega(step, total_steps, omega_min=25, omega_max=45): return omega_min + (omega_max - omega_min) * \ (1 - math.cos(math.pi * step / total_steps)) / 2 if step % 500 == 0: new_omega = caos_omega(step, total_steps) model.update_omega(new_omega)
```

Results:

Config	Omega Range	Final PSNR	vs Baseline	Status
SIREN A2 Baseline	Fixed 30	40.37 dB	—	Reference
CAOS-SIREN Run 1	10 → 60	29.77 dB	-10.60 dB	Failed
CAOS-SIREN Run 2	25 → 45	19.59 dB	-20.78 dB	Failed worse

Why it failed:

The PSNR curve shows a characteristic bell shape — rising then falling as ω increases. Two root causes: (1) The network was initialized for a specific ω but updating it mid-training invalidates the carefully derived initialization assumptions. (2) Interestingly, Run 1 (10→60) outperformed Run 2 (25→45) — spending more time at low frequencies allows the network to first capture global structure. However both runs underperform fixed $\omega=30$, confirming that SIREN's fixed initialization is already optimally tuned and dynamic scheduling disrupts it.

4.3 Improvement II — Modified Loss Functions

Attempt A — Fourier Frequency Loss (FFL):

We proposed adding an FFT magnitude loss to encourage high-frequency detail: $L_{\text{total}} = L_2 + \lambda * ||\text{FFT}(\text{pred}) - \text{FFT}(\text{gt})||_1$ with λ in $\{0.01, 0.05, 0.1\}$.

```
def ffl_loss(pred, gt, lambda_ffl=0.05, step=0, warmup_steps=2000): l2 = ((pred - gt)**2).mean() lam = lambda_ffl * min(1.0, step / warmup_steps) if lam == 0: return l2 H, W = 512, 512 pred_fft = torch.fft.fft2(pred.reshape(H, W)) gt_fft = torch.fft.fft2(gt.reshape(H, W)) # Normalized FFT L1 loss fft_l1 = torch.abs(torch.abs(pred_fft) - torch.abs(gt_fft)).mean() / (H * W) return l2 + lam * fft_l1
```

Lambda	Final PSNR	vs Baseline	Final Loss
Baseline	40.37 dB	—	—
0.01	29.37 dB	-11.00 dB	0.0387
0.05	26.77 dB	-13.60 dB	0.1745
0.10	25.77 dB	-14.60 dB	0.3541

Failed: FFT magnitudes operate on much larger scale than L2 pixel loss — even small λ values cause the FFT term to dominate optimization. Higher λ = worse PSNR (consistent trend confirms scale mismatch). SIREN already captures frequencies through sine activations making explicit FFT supervision redundant.

Attempt B — Gradient Supervision Loss:

We proposed adding explicit gradient supervision using ground truth gradients available in the SIREN dataset: $L_{\text{total}} = L_2 + \lambda * ||\text{grad}(\text{pred}) - \text{grad}(\text{gt})||_1$

```
def gradient_loss(model_output, gt, lambda_grad=0.1, step=0, warmup_steps=1000): pred = model_output['model_out'] target = gt['img'] # L2 pixel loss l2_loss = ((pred - target)**2).mean() # Warmup lambda lam = lambda_grad * min(1.0, step / warmup_steps) if lam == 0: return l2_loss, l2_loss.item(), 0.0 # Predicted gradients via autograd pred_grad = diff_operators.gradient(model_output['model_out'], model_output['model_in']) # Ground truth gradients from dataset gt_grad = gt['gradients'] # L1 gradient loss grad_loss = torch.abs(pred_grad - gt_grad).mean() total = l2_loss + lam * grad_loss return total, l2_loss.item(), grad_loss.item()
```

Lambda	Final PSNR	SSIM	vs Baseline	Training Stable?
Baseline	40.37 dB	—	—	Yes
0.01	26.14 dB	0.59	-14.23 dB	Yes
0.05	21.48 dB	0.38	-18.89 dB	Yes
0.10	21.47 dB	0.31	-18.90 dB	Yes

Most stable of all improvements — smooth rising PSNR curves, no instability. $\lambda=0.01$ best at 26.14 dB. Failed because gradient supervision competes with pixel reconstruction — network cannot simultaneously minimize both. Most promising direction for future work.

5. Complete Results Summary

5.1 All Experiments

Experiment	Config	PSNR/MSE	vs Baseline	Verdict
A2: SIREN Image Fixed omega=30, 10K steps		40.37 dB	Reference	Pass
A2: ReLU P.E.	NeRF encoding	31.11 dB	-9.26 dB	Pass
A2: RBF-ReLU	1024 RBF input	23.50 dB	-16.87 dB	Pass*
A2: ReLU	Standard ReLU	23.99 dB	-16.38 dB	Pass
A2: Tanh	Tanh activation	21.91 dB	-18.46 dB	Pass
A2: Bach SIREN	Audio MSE	~0 (converged)	—	Pass
A2: Bach ReLU	Audio MSE	2.48e-2	—	Pass
A2: Video SIREN	5K steps	30.70 dB	+0.80 vs paper	Pass!
A2: Video ReLU	5K steps	19.81 dB	-5.31 vs paper	Partial
A3: CAOS Run1	omega 10 to 60	29.77 dB	-10.60 dB	Fail
A3: CAOS Run2	omega 25 to 45	19.59 dB	-20.78 dB	Fail
A3: FFL 0.01	FFT loss	29.37 dB	-11.00 dB	Fail
A3: FFL 0.05	FFT loss	26.77 dB	-13.60 dB	Fail
A3: FFL 0.10	FFT loss	25.77 dB	-14.60 dB	Fail
A3: Grad 0.01	Gradient L1	26.14 dB	-14.23 dB	Fail
A3: Grad 0.05	Gradient L1	21.48 dB	-18.89 dB	Fail
A3: Grad 0.10	Gradient L1	21.47 dB	-18.90 dB	Fail

5.2 Key Findings

1. SIREN consistently outperforms all baselines across image, audio, and video — confirming the paper's central claim.
2. Audio results reproduced with exceptional accuracy — ReLU MSE within 1% of paper values.
3. Video SIREN exceeded paper PSNR (30.70 vs 29.90 dB) despite only 1.8% of training steps — fast convergence confirmed.
4. All proposed improvements (CAOS, FFL, GradLoss) resulted in lower PSNR than baseline — revealing that SIREN's fixed initialization is already optimal.
5. SIREN's fixed omega=30 initialization is more important than training dynamics — changes to omega schedule break the carefully derived math.
6. Gradient loss was most stable improvement — smooth training curves unlike CAOS instability.

7. FFL confirmed scale mismatch issue — FFT magnitudes incompatible with L2 scale.
8. Negative results are scientifically valid — they confirm SIREN's initialization scheme is a tightly optimized system.

5.3 Core Insight

All improvement failures point to the same conclusion: SIREN's strength comes from its mathematically principled design — the initialization scheme, fixed omega, and pure L2 loss form a tightly coupled system. Modifying any one component disrupts the others. Future improvements should focus on architectural changes (like H-SIREN which modifies the activation function itself) rather than training dynamics or loss modifications.

6. Conclusion

We successfully reproduced the key findings of the SIREN paper across image, audio, and video experiments. The core claim — that periodic sine activations significantly outperform ReLU for implicit neural representations — is confirmed by all our experiments. Audio reproduction was particularly accurate (within 1%), and video SIREN actually exceeded the paper's PSNR despite training for only 1.8% of the required steps.

For Assignment 3, all three proposed improvements (CAOS omega scheduling, FFT frequency loss, and gradient supervision loss) resulted in lower PSNR than the baseline. Rather than being a failure, this reveals an important insight: SIREN's tightly coupled design of initialization + fixed omega + L2 loss is already mathematically optimal. Improvements to SIREN should target architectural modifications rather than training dynamics.

7. References

- [1] Sitzmann, V., Martel, J.N.P., Bergman, A.W., Lindell, D.B., & Wetzstein, G. (2020). Implicit Neural Representations with Periodic Activation Functions. NeurIPS 2020. arXiv:2006.09661
- [2] Official GitHub Repository: <https://github.com/vsitzmann/siren>
- [3] Liu, Z., et al. (2024). FINER: Flexible spectral-bias tuning in Implicit NEural Representation by Variable-periodic Activation Functions. CVPR 2024.
- [4] Gao, R., & Jaiman, R. (2024). H-SIREN: Improving Implicit Neural Representations with Hyperbolic Sine. arXiv:2410.04716
- [5] Gu, J., et al. (2021). Focal Frequency Loss for Image Reconstruction and Synthesis. arXiv:2012.12821
- [6] Martin, D., et al. (2001). A Database of Human Segmented Natural Images. ICCV 2001. (BSDS500 dataset)
- [7] Mildenhall, B., et al. (2020). NeRF: Representing Scenes as Neural Radiance Fields. arXiv:2003.08934